

Fireblocks ERC20F Differential Audit



September 24, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Privileged Roles	6
Trust Assumptions	7
Low Severity	9
L-01 Upgrading Live V1 Proxies Permanently Disables The decimals Function	9
L-02 Documented Decimals Immutability Is Not Enforced	10
L-03 Deterministic Token Address Is Coupled To Initializer Calldata	11
L-04 Lower-Decimal Initialization Breaks Bridge Compatibility And Token Accounting	12
Notes & Additional Information	13
N-01 Inconsistent Pragma Directives Across the Codebase	13
N-02 Missing Zero-Address Check On The Trusted Forwarder	14
N-03 Changed initialize Selector Breaks Existing Deployment Tooling	15
N-04 Lack of Security Contact	16
N-05 Design Document Diverges From The Implemented Storage Layout	17
N-06 Implementation Contracts Expose A Permanently Reverting Decimals Getter	18
Client Reported	19
CR-01 Spender Is Not Subject To Access Registry Checks In transferFrom	19
Conclusion	21
Appendix	22
Issue Classification	22

Summary

Type	DeFi	Total Issues	11 (4 resolved)
Timeline	From 2026-08-27 To 2026-08-27	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	4 (1 resolved)
		Notes & Additional Information	6 (3 resolved)
		Client Reported Issues	1 (0 resolved)

Scope

OpenZeppelin performed a differential audit of the [fireblocks/fireblocks-smart-contracts](#) repository at commit [02ca896](#). The review covered the changes introduced by two commits: [8bf2fc6](#), which grants the `BURNER_ROLE` at initialization, and [02ca896](#), which makes the token decimals configurable. Pre-existing behavior of the token templates was reviewed only insofar as these changes interact with it.

Update: All fixes applied by the Fireblocks team in response to this audit were merged into the `main` branch at commit [4867453](#), which incorporates pull requests [#40](#) to [#43](#). The fix review was performed against that commit.

In scope were the following files:

```
contracts
├── gasless-contracts
│   └── ERC20FGasless.sol
├── library
│   ├── Errors
│   └── LibErrors.sol
└── ERC20F.sol
```

System Overview

`ERC20F` is an upgradeable ERC-20 token template that combines standard fungible token behavior with administrative controls that are commonly relevant for permissioned or compliance-sensitive deployments. The reviewed changes extend its initializer twice. First, the initializer requires a nonzero burner address and grants the `BURNER_ROLE` at deployment, removing the separate post-deployment grant that a redemption-bearing token previously needed. Second, the token stores a deployment-specific decimals value instead of relying on the ERC-20 default of 18. That value is supplied to the initializer, validated only against an upper bound of 18 so that any value from 0 through 18 is accepted, and written once with no setter exposed afterwards. The `LibErrors` library is in scope because it declares the `InvalidDecimals` error that the new validation and the decimals accessor revert with.

The decimals value is held in ERC-7201 namespaced storage rather than in a sequentially appended state variable, and it is stored offset by one so that an uninitialized slot remains distinguishable from a deliberately configured value of zero. The namespaced layout is deliberate: `ERC20FGasless` and the out-of-scope `ERC20FV2` upgrade contract both linearize additional inherited state directly after the `ERC20F` storage footprint, which a sequential slot would have displaced. The `decimals` function reverts rather than returning a default when the slot has never been written.

At a high level, `ERC20F` extends an ERC-20 with several control-plane subsystems exposed through inherited modules and local authorization hooks. The contract supports minting, burning, pausing, permit-based approvals, token recovery from accounts that fail an external access check, upgradeability through UUPS, and optional access gating through an external `accessRegistry`. Transfers, minting, burning, and recovery all pass through local checks that enforce pause state and, when an access registry is configured, sender or receiver eligibility.

`ERC20FGasless` builds on the same token logic and adds ERC-2771 meta-transaction support by overriding `_msgSender()` and `_msgData()` to trust a configured forwarder. Its initialization flow mirrors `ERC20F` while also setting the trusted forwarder. The gasless variant intentionally disables `multicall`, and it adds a dedicated authorization hook for trusted forwarder updates.

The reviewed code therefore exposes two principal operational surfaces. The first is the token control surface, including supply changes, recovery, pausing, upgrades, and optional access-

policy enforcement. The second is the gasless forwarding surface, where the correctness of relayed identity depends on a designated forwarder. These surfaces define the main security boundaries for the system.

Security Model and Trust Assumptions

Privileged Roles

- `DEFAULT_ADMIN_ROLE`
 - Administers every other role, and can therefore grant and revoke all privileges listed below.
 - Cannot revoke the role from itself, and cannot renounce the role, so the token cannot be left without an administrator through either of those paths.
- `MINTER_ROLE`
 - Can call `mint` to create new tokens for an allowed recipient.
- `BURNER_ROLE`
 - Can call `burn` to destroy tokens held by the caller.
- `RECOVERY_ROLE`
 - Can call `recoverTokens` to transfer tokens from an account that fails the configured access check, when an access registry is set.
- `PAUSER_ROLE`
 - Can authorize pause-related operations, which can halt transfers and several administrative actions guarded by `whenNotPaused`.
- `UPGRADER_ROLE`
 - Can authorize UUPS implementation upgrades through `_authorizeUpgrade`.
- `CONTRACT_ADMIN_ROLE`

- Can authorize updates to the access registry.
- Can authorize contract URI updates.
- In `ERC20FGasless`, can authorize trusted forwarder updates.

Trust Assumptions

- If an `accessRegistry` is configured, the token trusts that external contract as the policy oracle for whether an account may send or receive tokens.
- In practice, this means transfer admissibility and recovery eligibility depend on external logic outside the scoped contracts.
- When no registry is configured, these access checks are bypassed.
- The gasless variant trusts its configured ERC-2771 forwarder to supply the correct original sender and calldata context.
- A faulty or malicious trusted forwarder could cause the contract to treat relayed calls as originating from the wrong account.
- Any account holding upgrade authority is trusted to deploy compatible implementations that preserve storage layout and security properties.
- This is particularly relevant here because the token now stores its decimals value in dedicated ERC-7201 namespaced storage, and the gasless variant composes this contract with additional inherited state.
- Deployment and initialization are security-critical.
- The decimals value is written during initialization and no public setter exists to change it afterwards, so a mistaken initialization creates a permanent token configuration error rather than a recoverable runtime setting.
- The initializer enforces only the upper bound of 18. Selecting a value appropriate for the deployment, and one compatible with any bridge adapter or operational tooling that assumes a particular precision, is left to deployment configuration.
- Write-once behavior holds by convention rather than by construction, since the internal function that writes the value is `virtual` and carries no initialization guard.
- The reviewed changes assume a freshly deployed token.
- The namespaced decimals slot is zero on every proxy deployed before this feature existed, and no initializer or reinitializer in scope populates it for such a proxy.

- Directing an existing proxy at an implementation built from the reviewed code is therefore outside the design's assumptions, even though nothing in the upgrade authorization path prevents it.

Low Severity

L-01 Upgrading Live V1 Proxies Permanently Disables The decimals Function

The `ERC20F` contract stores a caller-chosen decimals value in ERC-7201 namespaced storage as the `decimalsPlusOne` variable, and the overridden `decimals` function [reverts with the `InvalidDecimals` error](#) whenever that slot is zero. Because the namespaced slot is untouched by every proxy deployed before this feature existed, any live V1 `ERC20F` proxy that is upgraded to an implementation built from the current code, whether `ERC20F`, `ERC20FGasless`, or `ERC20FV2`, reads a zero value the moment the upgrade completes.

The only writers of the `decimalsPlusOne` variable are the `_setDecimals` calls inside the `initialize` functions of [the `ERC20F` contract](#) and the `ERC20FGasless` contract, both guarded by the `initializer` modifier. A live proxy already sits at version 1, so neither can ever execute on it again, and no public setter or reinitializer exists to populate the value afterwards. The `initializeV2` function of the `ERC20FV2` contract, the designated migration for exactly this upgrade, [sets only the trusted forwarder](#) and additionally consumes the version 2 reinitializer slot. The `_authorizeUpgrade` function performs [only a role check](#), so nothing enforces the fresh-deploy-only assumption stated in the feature's commit message.

As a result, an operator who follows the documented gasless migration leaves the token with a `decimals` function that reverts on every call. Token transfers, mints, and burns keep working, but every consumer of the ERC-20 metadata interface breaks, including this repository's own `FungibleLayerZeroAdapter` contract, whose constructor [queries the token's decimals](#) and therefore can no longer be deployed for the upgraded token. Wallets, explorers, and integrations that scale amounts by the reported decimals fail alongside it. Recovery requires authoring and deploying a new implementation with a `reinitializer(3)` migration, since no remediation path exists in the shipped code.

Consider having the `initializeV2` function of the `ERC20FV2` contract call the `_setDecimals` function with a value of 18, matching the fixed default that V1 proxies exposed before the upgrade. Alternatively, consider having the `decimals` function return 18 when the `decimalsPlusOne` variable is zero, at the cost of losing the explicit-initialization invariant for fresh deployments. Furthermore, consider applying the same population step to any future migration that introduces new namespaced state consumed by inherited code.

Update: Acknowledged, not resolved. The Fireblocks statement for the issue:

Risk accepted.

The configurable-decimals change was scoped, in both the design document and the PR, to newly deployed tokens only. We will not upgrade existing live V1 proxies onto an implementation built from this code, so the unwritten-slot condition does not arise in our operational model.

L-02 Documented Decimals Immutability Is Not Enforced

During the initialization of a token based on the `ERC20F` contract, the `initialize` function configures the token decimals through the internal `_setDecimals` function, which writes the value offset by one into the ERC-7201 namespaced `ERC20FStorage` struct. The accompanying documentation declares that this is the single writer of the decimals value and that the value is immutable after `initialize`.

However, that immutability holds only by convention. The `_setDecimals` function carries no restriction on when it may be called, so any inheriting contract can expose a public setter that invokes it after initialization, or override it since it is `virtual`, silently voiding the documented guarantee. Off-chain systems, price feeds, and wrapping contracts routinely cache the result of the `decimals` function on the assumption that it never changes, so a post-initialization mutation would skew every cached or historical interpretation of token amounts. In addition, an override that omits the offset-by-one encoding would make a configured value of zero indistinguishable from an unset slot, causing the `decimals` function to revert or return an off-by-one value. Within the repository the guarantee currently holds, as the only other caller is the initializer of the `ERC20FGasless` contract, but nothing prevents an extension from breaking it.

Consider restricting the `_setDecimals` function with the `onlyInitializing` modifier so that any call outside an initializer reverts, making the documented write-once behavior compiler-enforced for all inheriting contracts rather than assumed.

Update: Resolved in [pull request #40](#) at commit [b943a91](#). The `_setDecimals` function now carries the `onlyInitializing` modifier, so any call made outside of an initializer or reinitializer reverts.

L-03 Deterministic Token Address Is Coupled To Initializer Calldata

When a token is deployed through the `deployDeterministic` function of the `GaslessFactory` contract, the CREATE2 address is derived from the concatenation of the creation code and the constructor inputs, not from the salt alone. Since the `Proxy` constructor takes the implementation address and the initialization calldata, the complete ABI-encoded `initialize` call sits inside the address preimage.

This derivation was previously insensitive to the token's operational configuration, because the burner was granted `BURNER_ROLE` after deployment and post-deployment configuration does not feed the hash. The recent changes added `burner` and `decimals_` to the `initialize` function of the `ERC20F` contract and its `ERC20FGasless` counterpart, making both values address-determining. As a result, any address precomputed under the previous encoding no longer corresponds to what a deployment produces, and, more durably, an issuer who legitimately needs a different burner account or a different precision on another chain loses the single-address-across-chains property that deterministic deployment exists to provide. The workaround, deploying byte-identical calldata everywhere and rotating the burner afterwards with `grantRole` and `revokeRole`, reintroduces the post-deployment grant that moving the grant into `initialize` was intended to eliminate.

Consider decoupling the address derivation from token configuration by deploying the `Proxy` contract with empty `_data` and routing the `initialize` call through the `postConfig` array of the `deployDeterministic` function, which already executes atomically after `Create2.deploy` and so does not affect the address. Alternatively, consider documenting in the NatSpec of the `deployDeterministic` and `computeAddress` functions that the deterministic address depends on the complete constructor input and that cross-chain address parity requires byte-identical initializer calldata, including the `burner` and `decimals_` values, on every chain.

Update: Acknowledged, not resolved. The Fireblocks statement for the issue:

Risk accepted.

We acknowledge that adding `burner` and `decimals_` to the initializer makes both values part of the CREATE2 address derivation, so a deterministic address is now a function of the full initializer calldata.

Deterministic address parity holds for deployments that use the same template and byte-identical initializer calldata. We will make customers aware that a deterministic address is only reproducible under those conditions, and that a different burner or

precision changes the resulting address. As noted in our L-01 response, this version of the token is a new deployment target and is not compatible with, or upgradeable from, V1, so there is no existing-deployment parity to preserve here.

L-04 Lower-Decimal Initialization Breaks Bridge Compatibility And Token Accounting

During the initialization of a token based on the `ERC20F` contract, the `initialize` function validates the caller-supplied precision only against an upper bound, reverting with the `InvalidDecimals` error when `decimals_` exceeds the `MAX_DECIMALS` constant of 18. Every value from 0 through 18 is accepted and stored, and because no setter exists, the chosen precision is fixed for the life of the token. The `ERC20FGasless` contract applies the same unbounded check.

This omission leaves two operational invariants unenforced. First, the `FungibleLayerZeroAdapter` constructor forwards the token's decimals value into the pinned `@layerzerolabs/oft-evm` implementation, whose constructor reverts with the `InvalidLocalDecimals` error whenever the local precision is below its shared precision of 6. A token initialized with 0 through 5 decimals therefore cannot be paired with the repository's bridge adapter, and since the precision cannot be changed after initialization, the only remedy is redeployment. Second, the `mint` and `burn` functions apply the supplied amount directly without normalizing it against `decimals()`, so a caller who follows Fireblocks' operational examples of one token as `1e18` raw units on a token configured with 6 through 17 decimals creates or destroys `10 ** (18 - decimals())` times the intended displayed amount.

The mismatch might also corrupt cross-chain supply. When a bridge transfer is initiated, the adapter's `_debit` function burns the full dust-adjusted local amount before the pinned `OFTCore` conversion narrows the shared-decimal amount to a `uint64` value through an unchecked cast. When that amount exceeds the maximum `uint64` value, the encoded message carries only the low 64 bits, and the destination adapter mints the truncated amount. As a result, more tokens are burned on the source chain than are minted on the destination chain, and the difference is permanently destroyed.

Consider restricting initialization to 18 decimals until every operational workflow and the bridge adapter share a single precision assumption. Alternatively, consider enforcing a minimum of 6 decimals in both the `ERC20F` and `ERC20FGasless` initializers, deriving the `mint` and `burn` amounts from `decimals()` in the supporting tooling, and rejecting any bridge transfer whose shared-decimal amount would exceed the maximum `uint64` value before the burn in the

`_debit` function. Furthermore, consider providing a migration path for any deployment already initialized with an incompatible precision.

Update: Acknowledged, not resolved. The Fireblocks statement for the issue:

Risk accepted.

- *We are keeping the 0 to 18 range by design. Some customers will need fewer than 6 decimals (e.g. 4), so a floor of 6 would rule out valid configs.*
- *Trade-off accepted: a sub-6-decimal token cannot pair with the current LayerZero adapter, which reverts at deploy with `InvalidLocalDecimals`. This is a loud deploy-time failure on the Adapter side, no silent loss. We will document it as a known constraint and revisit shared-decimals handling in the adapter if sub-6 bridging is needed.*

Notes & Additional Information

N-01 Inconsistent Pragma Directives Across the Codebase

Throughout the codebase, the first-party contracts do not agree on a single Solidity compiler version. The token contracts and their supporting libraries, such as the `ERC20F` contract, [pin the compiler to version 0.8.36](#), while the `bridge-adapter`, `vaults`, and `mocks` trees, such as the `VestingVault` contract, [pin the compiler to version 0.8.29](#). In addition, the `LibErrors` library is the only first-party file that [declares a floating pragma](#) of `^0.8.29` instead of an exact version.

Mixing exact pragma versions means that different parts of the protocol are compiled and deployed with different compiler releases, so compiler-specific behavior and known compiler defects have to be tracked against two versions instead of one. The floating pragma in the `LibErrors` library additionally allows that file to be compiled with any future 0.8.x release, so the bytecode ultimately deployed for contracts importing it depends on the toolchain configuration at build time rather than on the source code alone. This weakens reproducibility of builds and makes verification against the audited artifacts harder, although it does not create

a direct vulnerability because all declared ranges are satisfied by the currently configured compilers.

Consider pinning the `LibErrors` library to the same exact compiler version used by the contracts that import it. Furthermore, consider unifying the whole codebase on a single exact pragma version, upgrading the files currently pinned to version 0.8.29 to version 0.8.36 once they have been tested against it.

Update: Acknowledged, will resolve. The Fireblocks statement for the issue:

Acknowledged.

We acknowledge the inconsistent pragma directives. Unifying the codebase on a single exact version is not something we are taking in this cycle: the bridge-adapter, vault, and mock trees are pinned to `0.8.29`, and some of these (the adapters in particular) may need further changes, so raising them to `0.8.36` now would break compilation across the project. We have tracked the unification for a future change once those trees are ready to move and can be retested.

N-02 Missing Zero-Address Check On The Trusted Forwarder

During the execution of the `initialize` function of the `ERC20FGasless` contract, the deployer supplies the privileged role holders together with the `trustedForwarder` address that gives this token variant its meta-transaction capability. Before any state is written, the function [validates every role address against the zero address](#) and reverts with the `InvalidAddress` error when one of them is unset.

The `trustedForwarder` parameter is the only address argument that receives no such validation, and it is [forwarded unchecked](#) into the `__ERC2771ContextInitializableUpgradeable_init` function, which [stores it without inspection](#). A deployment that accidentally passes the zero address therefore completes successfully, yet produces a contract that is behaviorally indistinguishable from the plain `ERC20F` contract, since no caller can ever match the stored forwarder. The meta-transaction support that is the sole reason to choose the gasless variant is silently absent. Until an operator notices the misconfiguration and corrects it through the `updateTrustedForwarder` function, any transaction relayed through the intended forwarder is not unwrapped by the `_msgSender` function and executes with the forwarder itself as the effective caller, so relayed operations act on the forwarder's own state instead of the signing user's and fail to take effect as intended.

Consider extending the zero-address validation in the `initialize` function of the `ERC20FGasless` contract to also cover the `trustedForwarder` parameter, reverting with the `InvalidAddress` error, so that the gasless variant cannot be deployed without the capability that distinguishes it. Alternatively, if deploying without an initial forwarder is meant to be supported, consider documenting this explicitly in the function's docstring so that integrators do not mistake the omission for an oversight.

Update: Acknowledged, will resolve. The Fireblocks statement for the issue:

Acknowledged.

The missing zero-address check on `trustedForwarder` predates the reviewed change set and is not introduced or altered by the configurable-decimals or burner-grant work. As this defect falls outside the scope of the changes under audit, we are not remediating it as part of this engagement and have tracked it in our internal backlog.

N-03 Changed `initialize` Selector Breaks Existing Deployment Tooling

When a new token instance is deployed behind a proxy, the deployment transaction ABI-encodes a call to the `initialize` function of the `ERC20F` contract and passes it as the proxy's initialization calldata. The recent changes extended this signature twice, first by adding the `burner` parameter so that the `BURNER_ROLE` is granted at initialization, and then by adding the `decimals` parameter for configurable decimals. The `initialize` function of the `ERC20FGasless` contract received the same extensions on top of its `trustedForwarder` parameter.

Because the parameter list changed, the function selector of `initialize` is different on both contracts. Any factory contract, deployment script, or operational runbook that still encodes the previous signature will produce calldata whose selector no longer matches a function on the new implementations, so the proxy's initializing delegatecall reverts and the entire deployment fails. The failure is loud rather than silent, and an already deployed proxy is unaffected because upgrading an implementation never re-runs the initializer, so the deviation cannot leave a proxy half-initialized. The impact is instead operational. Every deployment pipeline pinned to the previous encoding breaks until it is updated, and integrations that documented a separate post-deployment `grantRole` step for the `BURNER_ROLE` now describe a flow that no longer matches the initializer's behavior. The `disabled_initialize overload` in the `ERC20FGasless` contract also has to track the parent signature exactly on

every such change, since a mismatch would stop it from shadowing the inherited selector and would expose an initialization path that skips the trusted forwarder setup.

Consider updating every factory, deployment script, and operational document that encodes the `initialize` calldata of the `ERC20F` and `ERC20FGasless` contracts to the new seven and eight parameter signatures before rolling out the new implementations. Additionally, consider adding a deployment test that initializes both contracts through their proxies with the new encoding, and a test asserting that the disabled `initialize` overload of the `ERC20FGasless` contract reverts with the `FunctionDisabled` error, so that any future signature drift between the two contracts is caught automatically.

Update: Acknowledged, will resolve. The Fireblocks statement for the issue:

Acknowledged.

The changed `initialize` selector does not require a change to the audited contracts. The affected calldata is not in these contracts; it lives in the deployment template in our internal contract-registry. We will update our internal configuration and deployment runbook to use the new seven and eight parameter `initialize` signatures, so there is no collision between the previous implementation and this one. Existing deployed proxies are unaffected, as upgrading an implementation does not re-run the initializer.

N-04 Lack of Security Contact

Providing a specific security contact, such as an email address or ENS name, within a smart contract significantly simplifies the process for individuals to report a vulnerability they identify in the code. This practice lets the code owners dictate the communication channel for vulnerability disclosure, avoiding miscommunication or unreported findings from readers who do not know how to reach the team. Within the reviewed scope, [the `ERC20F` contract](#), [the `ERC20FGasless` contract](#), and [the `LibErrors` library](#) do not declare a security contact in their contract-level documentation.

The omission is inconsistent with the rest of the codebase, where the bridge-adapter contracts already [carry the `@custom:security-contact` tag](#) pointing to [support@fireblocks.com](#). As a result, a reader inspecting the verified source of a deployed token, the most likely place for an external researcher to encounter the code, finds no disclosure channel on precisely the contracts that hold user balances, while auxiliary contracts advertise one.

Consider adding a NatSpec comment containing a security contact above each contract and library definition. Using the `@custom:security-contact` convention is recommended, as it has been adopted by the OpenZeppelin Wizard and is already in use elsewhere in the codebase.

Update: Resolved in [pull request #41](#) at commit [11f1264](#).

N-05 Design Document Diverges From The Implemented Storage Layout

The client-supplied design document "Fireblocks ERC20F Token Configuration Design Decisions" (version 1.0, August 3, 2026) records the design intent behind the configurable decimals feature, and a reader consulting it for the storage design finds decision 06 ("Storage layout") and the accompanying "Contract Surface" section prescribing a sequentially appended state variable: "Declare `_decimals` in the token's own storage layer, above the inherited base storage" with the note that "the ERC20F layer holds no storage of its own, so `_decimals` occupy the first free slot with no collision", illustrated as `uint8 private _decimals;` returned directly by `decimals()`. The shipped `ERC20F` contract does not follow this. It instead [holds the value in ERC-7201 namespaced storage](#), read through a [dedicated slot accessor](#) rather than a sequential slot.

The implementation is the correct side of this divergence, and the contract's own NatSpec [explains why the namespaced layout was chosen](#): a sequentially appended slot, as decision 06 describes, would collide with the `trustedForwarder_` variable that the `ERC20FGasless` and `ERC20FV2` contracts linearize immediately after the `ERC20F` footprint through the `ERC2771ContextInitializableUpgradeable` mixin. The document therefore stands as a stale reference whose recommended approach the code deliberately rejects, and an engineer following decision 06 literally when building a future extension would reintroduce the exact storage collision the shipped layout avoids. A second, smaller divergence compounds this: the code [stores the value offset by one](#) so that an unset slot (zero) stays distinguishable from a configured value of zero decimals, whereas the "Contract Surface" listing stores and returns the raw value with no such encoding.

Separately, decision 05 ("Initializer signature and parameter ordering") closes by directing the reader to "see decision 07 for the deployment consequence" of the changed initializer signature, but the document contains no decision 07 and ends at decision 06. The deployment consequence it gestures at is real and is captured elsewhere in this review, yet within the document itself the cross-reference dangles and leaves the reader with no destination.

Consider updating decision 06 and the "Contract Surface" section of the design document to describe the ERC-7201 namespaced storage layout the code actually implements, including the offset-by-one encoding and the storage-collision rationale that motivated it, so the document cannot mislead a future extension into a sequential slot. Furthermore, consider correcting or removing the reference to the absent decision 07 in decision 05.

Update: Resolved. The Fireblocks team provided version 1.1 of the "Fireblocks ERC20F Token Configuration Design Decisions" document, where these items are corrected.

N-06 Implementation Contracts Expose A Permanently Reverting Decimals Getter

During any deployment of the `ERC20F` contract, the implementation must be placed on-chain before a proxy can point at it, and its constructor `calls _disableInitializers` so that the implementation address can never be initialized. However, the `decimals` function `reverts with the InvalidDecimals error` whenever the namespaced `decimalsPlusOne` variable is zero, and its only writer, the `_setDecimals` function, is reachable solely through the `initializer`-gated `initialize` function.

As a consequence, every correctly deployed `ERC20F` implementation, along with the `ERC20FGasless` and `ERC20FV2` contracts that inherit the same constructor and override unchanged, permanently reverts on `decimals` while every other public view answers normally. The docstring above the override states that the unset case "cannot occur for a token deployed through {initialize}", which is false for the implementation address, where the case is guaranteed and unrepairable. The impact is confined to off-chain surfaces, since the implementation holds no funds or roles and proxies are unaffected. A verified implementation renders `decimals` as an error row on block explorers, and indexers or ERC-20 conformance tooling probing the address see a partially answering contract.

Consider amending the docstring so that it no longer asserts an invariant the implementation address violates, stating instead that the revert is the permanent state of the implementation contract itself. Alternatively, consider having the `decimals` function fall back to `18` when the stored value is unset, restoring the behavior the contract had before the override was introduced, at the cost of making an uninitialized configuration silent again.

Update: Resolved in [pull request #42](#) at commit [9e8f0a8](#). The Fireblocks team [rewrote the docstring of the `decimals` function](#) so that it no longer claims the unset case cannot occur. Note that the `@notice` and `@return` tags of the function were removed in the process.

Consider restoring them so that the user-facing documentation of the `decimals` function remains consistent with the rest of the contract.

Client Reported

CR-01 Spender Is Not Subject To Access Registry Checks In `transferFrom`

This behavior was raised during a previous engagement and was accepted by the Fireblocks team as a design decision at the time, so it does not originate from the changes reviewed in this differential audit. During the fix review phase of this audit, the Fireblocks team informed that the decision had been revisited in light of tightening compliance expectations and submitted a fix, which is documented here for completeness.

The `ERC20F` contract consults the configured access registry on every token movement so that accounts denied by the registry cannot send or receive tokens. On the `transferFrom` path, however, [the checks covered only the `from` and `to` addresses](#), and the identity of the caller spending the allowance was never validated, whereas [the `transfer` function does validate the caller](#). As a result, an account denied by the registry could still move tokens between two allowed accounts by spending an allowance that had been granted to it before or after the denial, since nothing on the transfer path examined the spender itself.

In [pull request #43](#) at commit [e05dca3](#), the Fireblocks team reversed the previously accepted design decision and now treats the spender as a party subject to registry policy. The `transferFrom` function [performs an access check on the caller](#) before the existing checks on `from` and `to` and before the allowance is consulted, so a denied account can no longer move tokens through an existing allowance. The endpoint checks are unchanged, and the "Calling Conditions" documentation of the function was updated to list the new requirement. Token recovery is unaffected, because [the `recoverTokens` function moves tokens through the internal `\_transfer` function](#) and never reaches the new check, so recovering tokens from a denied account remains possible. The change is scoped to the `ERC20F` contract. The equivalent operator behavior on the transfer functions of the `ERC721F` and `ERC1155F` contracts was intentionally left unchanged and is tracked separately by the team for a future change.

Three aspects of the change are worth noting. First, a denied spender is rejected through [the `shared access helper`](#), which only distinguishes senders from receivers, so the revert reports

the `ERC20InvalidSender` error naming the spender even though the `IERC20Errors` interface already declares the `ERC20InvalidSpender` error for this case. Second, the new check applies to every account that spends an allowance, including integration contracts such as routers or escrows. Under an allowlist registry, such a contract must itself be listed or its calls to the `transferFrom` function revert even when both endpoints are allowed, and since the `ERC20FGasless` and `ERC20FV2` contracts inherit the function unchanged, a live token upgraded to an implementation built from this code adopts the new policy as part of that upgrade. Third, the `approve` and `permit` functions are not subject to access registry checks, so a denied account can still be granted allowances, which become spendable only if the registry later re-admits the account.

Conclusion

OpenZeppelin audited two changes to Fireblocks' `ERC20F` token templates: the grant of the `BURNER_ROLE` during initialization, and the introduction of a deployment-specific decimals value together with the mirrored initialization path in `ERC20Fgasless`. Both changes are confined to the initializer and its supporting storage and error definitions, and they sit alongside existing privileged control surfaces for minting, burning, pausing, recovery, upgrades, and gasless forwarding.

The burner change is a narrow one. It grants a role that was previously assigned in a separate post-deployment transaction, models the new parameter identically to the existing minter parameter, and does not widen what the role can do, as the `burn` function still only destroys tokens held by the caller. The decimals change is small in code but operationally significant, because it converts a fixed protocol constant into a per-deployment parameter that becomes immutable once set. Storing that value in ERC-7201 namespaced storage is a sound choice, since it keeps the new state clear of the sequential storage that the gasless variants extend.

The findings concentrate on the boundaries of that new parameter rather than on the mechanics of the changes. A medium-severity issue describes how bounding decimals only from above admits a precision below what the repository's own LayerZero bridge adapter requires, and how amounts that assume the previous fixed precision distort minting, burning, and cross-chain accounting. Two low-severity issues follow from the same design: upgrading an existing proxy leaves the namespaced slot unwritten and the `decimals` function permanently reverting, and the documented write-once guarantee rests on convention rather than on an initialization guard. The notes address the trusted forwarder, the changed initializer selector, pragma consistency, security contacts, and a stale design document.

None of these findings indicate a flaw in the added logic itself. Rather, they mark the assumptions the changes now rest on: that deployments are fresh rather than upgraded, that the chosen precision is validated against the integrations that consume it, and that deployment tooling is regenerated for the new initializer signature. Correctness in this architecture continues to depend on secure initialization, sound role administration, a trustworthy access-policy oracle when one is configured, and disciplined upgrade practices that preserve both storage safety and the intended control model.

The OpenZeppelin team thanks Fireblocks for its cooperation during this audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.